Shor's Algorithm

From number theory and period finding to circuits, CUDA-Q simulation and cryptography — explained step by step

CUDA-Q Quantum Algorithm Benchmarking Suite

Companion lecture to `Shor.ipynb`

All simulation results in these slides were computed with NVIDIA CUDA-Q 0.16.0 on a CPU (`qpp-cpu`, `density-matrix-cpu`; no GPU) and checked against exact integer arithmetic. No physical quantum processor was used.

What you will learn today (two-hour session)

1–3. **Factoring**: why it matters, the number theory you need, classical algorithms and their cost
4. The **core idea**: factoring reduces to finding the period of $a^x \bmod N$
5–8. The **quantum part**: registers, period finding, the QFT, phase estimation
9. **Continued fractions**: from a measured number to the period
10. **Worked examples**: $N = 15$ and $N = 21$, every step
11–13. **Circuits, CUDA-Q code and simulation results** of `Shor.ipynb`
14–16. **Limits, cryptography and research**: resources, RSA, post-quantum cryptography

Suggested timing: modules 1–4 (35 min), 5–9 (40 min), 10–13 (30 min), 14–16 (15 min).

You only need

integer division with remainder, complex numbers $e^{i\theta}$, 2×2 matrices.

Four kinds of statements

IN REPOSITORY code that exists in this repository

RECORDED RUN output stored in `Shor.ipynb`

THEORY ONLY explained, not implemented here

ILLUSTRATIVE EXPERIMENT small experiments run for this lecture (`shor_source/make_figures.py`)

What this repository actually contains

Topic	Status	Where
order-finding circuit: Hadamards, controlled $U_{a^{2^j}}$ as exact permutation matrices, inverse QFT	IN REPOSITORY	`Shor.ipynb`
continued fractions, multiples test, gcd post-processing, retries with new a, verification	IN REPOSITORY	`Shor.ipynb`
resource formulas ($3n$, $2n+3$, Gidney–Ekerå) and reverse estimation	IN REPOSITORY	`Shor.ipynb`
default run $N = 15$, $a = 7$ (12 qubits, 1000 shots, `qpp-cpu`)	RECORDED RUN	`Shor.ipynb`
further cases ($N = 15$ with $a = 2, 14$; $N = 21$; $N = 35$), exact distributions	ILLUSTRATIVE EXPERIMENT	`make_figures.py`
gate-level multiplier for $N = 15$, depolarizing noise, approximate QFT	ILLUSTRATIVE EXPERIMENT	`make_figures.py`
semiclassical QFT, Beauregard/Gidney–Ekerå arithmetic, error correction, hardware runs	THEORY ONLY	—

No GPU, multi-GPU or quantum-hardware run was performed. Qiskit and Qiskit Aer are *not* installed and not used; the notebook refuses hardware targets because its multipliers are matrix gates.

What is Shor's algorithm?

- A **quantum algorithm** (Peter Shor, 1994) that finds a non-trivial factor of an integer N in time **polynomial** in the number of digits $n = \lceil \log_2 N \rceil$.
- It is **hybrid**: a quantum computer solves only one sub-problem — **order (period) finding** — and a classical computer does the rest.
- The same idea solves the **discrete-logarithm** problem, including on elliptic curves.
- Consequence: RSA, Diffie–Hellman and elliptic-curve cryptography would be broken by a large **fault-tolerant** quantum computer.

In one line

factoring $\xrightarrow{\text{classical}}$ order finding $\xrightarrow{\text{quantum}}$ period r $\xrightarrow{\text{classical}}$ factors

Not to be confused with

the **QFT**: a subroutine inside Shor's algorithm, not the factoring algorithm itself; and **Grover**: a quadratic search speed-up.

History and Peter Shor's contribution

- **1985–1994**: Deutsch, Deutsch–Jozsa, Bernstein–Vazirani and **Simon (1994)** show quantum speed-ups for artificial problems; Simon's algorithm finds a hidden *XOR*-period.
- **1994**: Shor generalises the idea to periods of a^x mod N and gives polynomial algorithms for **factoring** and **discrete logarithms** (FOCS 1994; SIAM J. Computing 1997). Coppersmith introduces the (approximate) QFT circuit the same year.
- **1995–1996**: Kitaev's phase-estimation view; Shor and Steane invent **quantum error correction**, showing large computations can in principle be protected.
- **2001–2012**: small demonstrations (factoring 15 with NMR, 21 with photons) — all with circuits *compiled using knowledge of the answer*.
- **2019–2025**: detailed resource estimates for RSA-2048 (Gidney–Ekerå 2019: $\approx$ 20 million noisy qubits; Gidney 2025: $<$ 1 million).

Status today

No quantum computer has factored a cryptographically relevant number. Demonstrations so far are tiny and partly "compiled".

Factoring, prime factorisation and why they matter

- **Factoring**: given composite N, find any $1 < p < N$ with $p \mid N$. Example: $91 \to 7$.
- **Prime factorisation**: the complete product of primes, $360 = 2^3 \cdot 3^2 \cdot 5$ — obtained by factoring repeatedly (primality tests are fast).
- Multiplying is easy, factoring seems hard: $p \cdot q$ takes microseconds for 1000-digit numbers; finding p, q from the product is believed infeasible classically.

Why it matters

- the security of **RSA** rests on it
- it is a natural problem in **BQP** not known to be in P
- it is the flagship example of an **exponential** quantum speed-up over the best known classical algorithms

Factoring is *not* believed to be NP-complete; Shor's algorithm does not imply that quantum computers solve NP-complete problems.

Classical versus quantum factoring

	Best classical (GNFS)	**Shor (fault-tolerant QC)**
cost for n-bit N	$\exp\left(c\, n^{1/3}(\log n)^{2/3}\right)$, sub-exponential	$O(n^3)$ gates (schoolbook), $O(n^2 \log n)$ with fast arithmetic
RSA-2048	far beyond any existing computer	$\approx$ hours to days on a hypothetical machine with 10^6–10^7 physical qubits
record	RSA-250 (829 bits), 2020, $\approx$ 2700 core-years	15, 21 (compiled demonstrations)
deterministic?	yes (heuristic run time)	no — probabilistic, repeat a few times

Where the speed-up comes from

Not from "trying all x at once": a measurement returns only one value. The speed-up comes from **interference** (QFT) that turns the periodicity of $a^x \bmod N$ into a measurable **frequency**.

Integers, primes, composites and the gcd

- **Prime**: exactly two divisors $(2, 3, 5, 7, 11, \dots)$. **Composite**: more $(15 = 3 \cdot 5)$.
- Every integer > 1 has a **unique prime factorisation**.
- $\gcd(a, b)$: the largest common divisor; a, b **coprime** if $\gcd = 1$.
- **Euclid**: $\gcd(a, b) = \gcd(b, a \bmod b)$, $O(\log N)$ steps — fast even for 2048-bit numbers.

Example: $\gcd(1071, 462)$

$$1071 = 2 \cdot 462 + 147$$
$$462 = 3 \cdot 147 + 21$$
$$147 = 7 \cdot 21 + 0 \quad \Rightarrow \quad \gcd = 21$$

Role in Shor

two gcds at the very beginning ($\gcd(a, N)$) and at the very end ($\gcd(a^{r/2} \pm 1, N)$) — both classical.

Modular arithmetic and congruences

- $a \bmod N$ = remainder of a divided by N, in $\{0, \ldots, N-1\}$ (clock arithmetic).
- $a \equiv b \pmod N$ iff $N \mid a - b$. Example: $17 \equiv 2 \pmod{15}$, $14 \equiv -1 \pmod{15}$.
- Congruences can be added and multiplied: $(a \cdot b) \bmod N = ((a \bmod N)(b \bmod N)) \bmod N$.
- **Inverse**: $b = a^{-1}$ with $ab \equiv 1$ exists iff $\gcd(a, N) = 1$. Example: $7 \cdot 13 = 91 \equiv 1 \pmod{15}$.

Multiplication table mod 15 (row 7)

y	1	2	4	7	8	11	13	14
$7y \bmod 15$	7	14	13	4	11	2	1	8

multiplying by 7 **permutes** the coprime residues

This permutation property is exactly what makes "multiply by a" a *reversible* (unitary) quantum gate.

Euler's totient and the multiplicative order

- **Euler's totient** $\varphi(N)$: number of $1 \le k \le N$ coprime to N. $\varphi(p) = p - 1$, $\varphi(pq) = (p-1)(q-1)$: $\varphi(15) = 8$, $\varphi(91) = 72$.
- **Euler's theorem**: $a^{\varphi(N)} \equiv 1 \pmod{N}$ for $\gcd(a, N) = 1$.
- **Order** $r = \mathrm{ord}_N(a)$: the **smallest** $r > 0$ with $a^r \equiv 1$. It divides $\lambda(N)$, which divides $\varphi(N)$.
- $a^x \bmod N$ is **periodic** with period r: $a^{x+r} \equiv a^x$.

Orders modulo 15

a	1	2	4	7	8	11	13	14
r	1	4	2	4	4	2	4	2

all divide $\lambda(15) = \mathrm{lcm}(2, 4) = 4$, none equals $\varphi(15) = 8$

The hard problem

Computing r for a 2048-bit N is believed to be as hard as factoring N.

Modular exponentiation by repeated squaring

Compute $a^x \bmod N$ with $O(\log x)$ multiplications instead of $x - 1$:

$$a^x = \prod_j \left(a^{2^j}\right)^{x_j}, \qquad x = \sum_j x_j 2^j.$$

1. square repeatedly: $a, a^2, a^4, a^8, \ldots \pmod N$
2. multiply the powers whose bit $x_j = 1$

Cost: $O(n)$ multiplications of n-bit numbers $= O(n^3)$ bit operations — **polynomial**.

$3^{13} \bmod 17$, $13 = 1101_2$

$3^1 = 3$, $3^2 = 9$, $3^4 = 81 \equiv 13$,
$3^8 \equiv 169 \equiv 16$
$3^{13} = 3^8 \cdot 3^4 \cdot 3^1 \equiv 16 \cdot 13 \cdot 3 = 624 \equiv \mathbf{12}$

Quantum version

counting qubit j controls a multiplication by the *classically precomputed* constant $a^{2^j} \bmod N$ — the same structure.

Number theory toolbox: three facts Shor needs

1. **Chinese remainder theorem**: for $N = pq$, a residue mod N is the same as a pair (residue mod p, residue mod q). Example: $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$ $\iff$ $x \equiv 8 \pmod{15}$.
2. **Square roots of 1**: modulo a prime only ± 1; modulo pq there are **four**: ± 1 and two *non-trivial* ones. For 15: $1, 4, 11, 14$.
3. **A non-trivial square root factors N**: if $x^2 \equiv 1$, $x \not\equiv \pm 1$, then $N \mid (x-1)(x+1)$ but divides neither factor, so $\gcd(x-1, N)$ and $\gcd(x+1, N)$ are non-trivial. Example: $x = 4$: $\gcd(3, 15) = 3$, $\gcd(5, 15) = 5$.

Bridge to Shor

If the order r of a is even, $x = a^{r/2}$ satisfies $x^2 = a^r \equiv 1$. If moreover $x \not\equiv -1$ (and $x \not\equiv 1$ holds automatically because r is minimal), x is a non-trivial square root of $1 \Rightarrow$ factors.

Trial division and simple methods

Trial division: test $d = 2, 3, 4, \ldots$ up to $\sqrt{N}$.

- $N = 10403$: stops at $d = 101$ $(10403 = 101 \cdot 103)$
- cost $O(\sqrt{N}) = O(2^{n/2})$: **exponential** in the bit length n
- 2048-bit N: $\approx 2^{1024}$ divisions — impossible

Pollard's rho finds a factor p in $\approx \sqrt{p} \leq N^{1/4}$ steps; **ECM** is sub-exponential in the size of p (good for small factors).

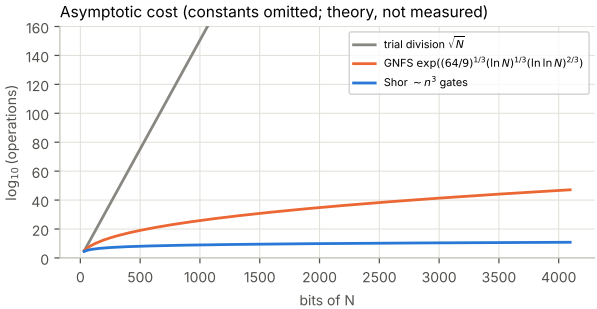
Careful

Not all classical methods are exponential! The **quadratic sieve** and the **general number field sieve** are **sub-exponential**.

Classically easy cases (checked first in the notebook)

even N; prime N (Miller–Rabin); prime powers $N = p^k$ (integer roots).

Exponential, sub-exponential, polynomial



THEORY ONLY asymptotic formulas with constants omitted — not measured run times.

method	cost (n bits)
trial division	$2^{n/2}$
Pollard rho	$2^{n/4}$
GNFS	$e^{1.92\, n^{1/3} (\ln n)^{2/3}}$
Shor	$O(n^3)$

Doubling n multiplies Shor's gate count by 8; for GNFS the cost grows faster than any polynomial.

Why RSA relies on factoring

1. choose primes p, q; publish $N = pq$ and e
2. secret $d = e^{-1} \bmod \varphi(N)$, $\varphi(N) = (p-1)(q-1)$
3. encrypt $c = m^e \bmod N$, decrypt $m = c^d \bmod N$

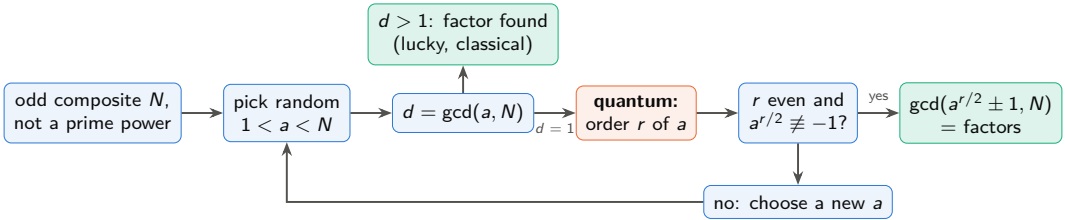
Knowing $p, q \Rightarrow \varphi(N) \Rightarrow d$: **factoring breaks RSA**.

Toy RSA

$p = 5,\ q = 11,\ N = 55,\ \varphi = 40,\ e = 7$
$d = 23$ $(7 \cdot 23 = 161 \equiv 1 \bmod 40)$
$m = 2$: $c = 2^7 \bmod 55 = 18$;
$18^{23} \bmod 55 = 2$ ✓

Real RSA uses ≥ 2048-bit N with padding; the toy example only shows the algebra.

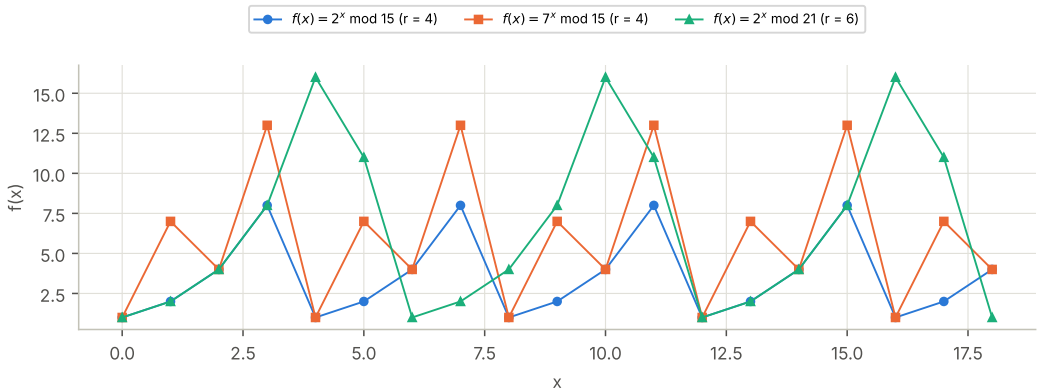
Reduction of factoring to order finding



Only one step is quantum

Everything except "order r of a" runs on a classical computer in polynomial time. For random a the test succeeds with probability $\geq 1 - 2^{-(k-1)} \geq \frac{1}{2}$ (k = number of distinct prime factors), so a few repetitions suffice.

The periodic function $f(x) = a^x \bmod N$



$2^x \bmod 15$: $1, 2, 4, 8, 1, 2, 4, 8, \ldots \Rightarrow r = 4$

$7^x \bmod 15$: $1, 7, 4, 13, 1, \ldots \Rightarrow r = 4$

$2^x \bmod 21$: $1, 2, 4, 8, 16, 11, 1, \ldots \Rightarrow r = 6$

The values look random inside one period; only the repetition carries information.

Why r must be even and $a^{r/2} \not\equiv -1$

$a^r \equiv 1 \Rightarrow (a^{r/2}-1)(a^{r/2}+1) \equiv 0 \pmod N$ — needs r **even**.

- $a^{r/2} \not\equiv 1$ always (else the order would be $r/2$)
- if $a^{r/2} \equiv -1$: $N \mid a^{r/2}+1$, gcds N and 1 — **trivial**
- otherwise N divides the product but neither factor $\Rightarrow$

$$p = \gcd(a^{r/2}-1, N), \qquad q = \gcd(a^{r/2}+1, N)$$

Cases

N, a	r	$a^{r/2}$	result
$15, 2$	4	4	3×5 ✓
$15, 14$	2	$14 \equiv -1$	trivial ✗
$21, 4$	3	—	odd ✗
$21, 5$	6	$20 \equiv -1$	trivial ✗
$21, 2$	6	8	7×3 ✓
$21, 6$	—	—	gcd $= 3$ (lucky)

A failed attempt is a failure: report it and repeat with another a.

When the reduction does not apply

- N **even**: factor 2 immediately.
- N **prime**: nothing to factor (fast primality tests).
- $N = p^k$ **a prime power**: $\mathbb{Z}_{p^k}^*$ is cyclic, so the only square roots of 1 are ± 1 and $\gcd(a^{r/2} \pm 1, N)$ is always trivial. Detected classically by integer k-th roots (`prime_power` in the notebook).
- N **with** $k \geq 2$ **distinct odd prime factors**: at least a fraction $1 - 2^{-(k-1)}$ of the bases works.

IN REPOSITORY `Shor.ipynb`

handles even N, primes and prime powers classically before any quantum computation; invalid or lucky bases are reported and replaced by a random a; failed attempts are retried up to `MAX_ATTEMPTS` times.

Qubits and registers — in the language of Shor

- A register of m qubits holds a superposition of the integers $0, \ldots, 2^m - 1$: $\sum_x \alpha_x |x\rangle$.
- **Counting (input) register**: t qubits for the exponent x; in the notebook $t = 2n$.
- **Work (function) register**: n qubits for values mod N, initialised to $|1\rangle$.
- Total: $t + n = 3n$ qubits; $n =$ `N.bit_length()`.

Register sizes (notebook)

N	n	t	qubits
15	4	8	12
21	5	10	15
33, 35	6	12	18
143	8	16	24

Integers are stored in binary: $|0001\rangle = |1\rangle$; the notebook puts the least significant bit on the *last* work qubit.

Superposition, Hadamard gates and quantum parallelism

$$H|0\rangle = \tfrac{1}{\sqrt{2}}(|0\rangle+|1\rangle), \qquad H^{\otimes t}|0\rangle^{\otimes t} = \tfrac{1}{\sqrt{2^t}}\sum_{x=0}^{2^t-1}|x\rangle$$

A reversible circuit U_f then gives

$$\tfrac{1}{\sqrt{2^t}}\sum_x |x\rangle|f(x)\rangle \quad (\text{“all } x \text{ at once”})$$

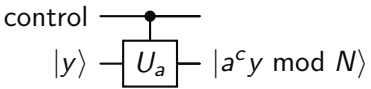
Parallelism is not enough

Measuring this state returns *one* random pair $(x, f(x))$ — no better than one classical evaluation. The period is hidden in the **amplitudes**; extracting it needs **interference**.

$t = 8$: each $|x\rangle$ has amplitude $1/16$, probability $1/256$.

Unitary, reversible and controlled gates

- Gates are **unitary**: $U^\dagger U = I$, hence **reversible**.
- $x \mapsto a^x \bmod N$ is **not** injective (x and $x + r$ collide) $\Rightarrow$ keep x: $|x\rangle\,|1\rangle \mapsto |x\rangle\,|a^x \bmod N\rangle$.
- $U_a\,|y\rangle = |ay \bmod N\rangle$ ($y < N$) is a **permutation** for $\gcd(a, N) = 1 \Rightarrow$ unitary.
- **Controlled-U**: apply U only on the $|1\rangle$ branch of a control qubit.



Phase gate $R_1(\lambda) = \mathrm{diag}(1, e^{i\lambda})$: $R_1(\pi) = Z$, $R_1(\frac{\pi}{2}) = S$, $R_1(\frac{\pi}{4}) = T$. Controlled R_1 gates build the QFT.

Entanglement, measurement and interference

Entanglement: after modular exponentiation the registers are correlated,

$$\frac{1}{\sqrt{8}}\sum_{x=0}^{7}|x\rangle\,|2^x \bmod 15\rangle = \frac{1}{\sqrt{8}}\big(\,|0\rangle\,|1\rangle+|1\rangle\,|2\rangle+|2\rangle\,|4\rangle+\dots\big)$$

Measurement of the work register with result 4 ($P = 1/4$) leaves $\frac{1}{\sqrt{2}}(|2\rangle + |6\rangle)$: periodic, period 4, random offset.

Interference: amplitudes of paths add *before* squaring:

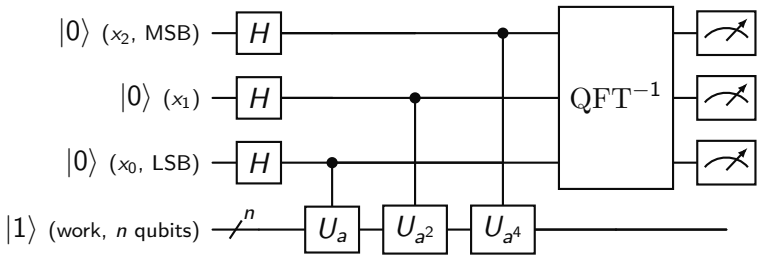
$$\left|\tfrac{1}{2} + \tfrac{1}{2}e^{i\theta}\right|^2 = \cos^2\tfrac{\theta}{2}$$

constructive for $\theta = 0$, destructive for $\theta = \pi$. The inverse QFT arranges the phases so that only $y \approx k2^t/r$ survive.

Implicit measurement

Measuring the work register is not necessary: the distribution of y is the same (the notebook measures only the counting register).

The period-finding (order-finding) circuit



1. Hadamards: uniform superposition over the exponent x ($t = 3$ shown; the notebook uses $t = 2n$)
2. controlled $U_{a^{2^j}}$ on qubit x_j: together $|x\rangle\,|1\rangle \mapsto |x\rangle\,|a^x \bmod N\rangle$ (repeated squaring)
3. inverse QFT on the counting register, measure y; classical post-processing gives r

State after each step

$$|\psi_0\rangle = |0\rangle^{\otimes t} |1\rangle$$

$$|\psi_1\rangle = \frac{1}{\sqrt{2^t}} \sum_{x=0}^{2^t-1} |x\rangle |1\rangle \qquad \text{(Hadamards)}$$

$$|\psi_2\rangle = \frac{1}{\sqrt{2^t}} \sum_{x=0}^{2^t-1} |x\rangle |a^x \bmod N\rangle = \frac{1}{\sqrt{2^t}} \sum_{v} \Big(\sum_{x:\, a^x \equiv v} |x\rangle \Big) |v\rangle \qquad \text{(group by value)}$$

$$|\psi_3\rangle = \sum_{y} \Big(\frac{1}{2^t} \sum_{x} e^{-2\pi i x y / 2^t} |y\rangle \Big) |a^x\rangle \qquad \text{(inverse QFT)}$$

For each value v the x's form an arithmetic progression $x_0, x_0 + r, x_0 + 2r, \ldots$ The geometric sum $\sum_m e^{-2\pi i m r y / 2^t}$ is large only when $ry/2^t$ is close to an integer, i.e. $y \approx k \cdot 2^t / r$. The offset x_0 only adds a phase.

When r divides 2^t: exact peaks

If $r \mid 2^t$, each progression has exactly $M = 2^t/r$ terms and

$$\left| \sum_{m=0}^{M-1} e^{-2\pi i m r y / 2^t} \right|^2 = \begin{cases} M^2 & y = k2^t/r \\ 0 & \text{otherwise} \end{cases}$$

so $P(y = k2^t/r) = 1/r$ for $k = 0, \ldots, r-1$.

$N = 15$, $a = 7$ (or 2), $t = 8$

$r = 4$: outcomes $0, 64, 128, 192$, each with probability exactly $1/4$.

$N = 15$, $a = 11$

$r = 2$: outcomes $0, 128$, each $1/2$.

For $r \nmid 2^t$ (e.g. $N = 21$, $r = 6$) the peaks have finite width — next modules.

Intuition: a quantum "frequency analyser"

- The counting register holds a **comb** with spacing r: $|x_0\rangle + |x_0 + r\rangle + |x_0 + 2r\rangle + \cdots$
- The Fourier transform of a comb with spacing r is a comb with spacing $2^t/r$ — like a discrete Fourier transform of a periodic signal.
- Measuring after the transform samples a random **frequency** k/r.

Why this is fast

The QFT on t qubits needs only $O(t^2)$ gates, while the classical FFT of the 2^t amplitudes would need $O(t\,2^t)$ operations — but the quantum version gives only *samples*, which is all we need.

The quantum Fourier transform

$$\mathrm{QFT}\,|x\rangle = \frac{1}{\sqrt{2^t}} \sum_{y=0}^{2^t-1} e^{2\pi i x y/2^t}\,|y\rangle$$

Product form (the key to an efficient circuit):

$$\mathrm{QFT}\,|x\rangle = \bigotimes_{l=1}^{t} \frac{|0\rangle + e^{2\pi i\, 0.x_{l-1}\cdots x_0}\,|1\rangle}{\sqrt{2}}$$

The inverse QFT has the conjugate phases, $e^{-2\pi i x y/2^t}$.

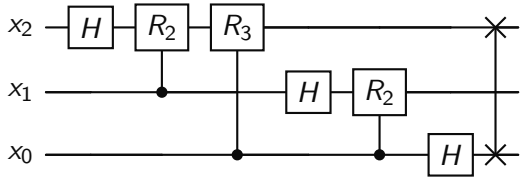
Why Shor needs it

It converts the **period** r of the comb into **peaks** at multiples of $2^t/r$ that a single measurement can sample.

Common mistake

The QFT is **not** Shor's algorithm. Without modular exponentiation and classical post-processing it factors nothing. See the companion QFT lecture for full details.

QFT circuit: Hadamards and controlled phase rotations



$R_k = R_1(2\pi/2^k)$. For t qubits: t Hadamards, $t(t-1)/2$ controlled phases, $\lfloor t/2 \rfloor$ SWAPs: $O(t^2)$ gates.

Inverse QFT in `Shor.ipynb` IN REPOSITORY

the mirror image: SWAPs first, then for each qubit the controlled phases `cr1(-pi/2**(k-j))` followed by `h`. For $t = 8$: 4 SWAPs, 28 controlled phases, 8 Hadamards.

From periodic states to Fourier peaks

Comb of M teeth, spacing r, offset x_0:

$$P(y) = \frac{1}{2^t M}\left|\sum_{m=0}^{M-1} e^{-2\pi i m r y/2^t}\right|^2 = \frac{\sin^2(\pi M r y/2^t)}{2^t M \sin^2(\pi r y/2^t)}$$

Peaks at $y \approx k2^t/r$; the **nearest** integer to each peak gets probability $\geq 4/\pi^2 \cdot \frac{1}{r}$.

Measurement outcome

$y/2^t \approx k/r$ with **random** $k \in \{0, \ldots, r-1\}$. Neither k nor r is known — only the ratio, approximately.

$N = 21$, $a = 2$, $t = 10$

$2^{10}/6 = 170.67$: peaks near $0, 171, 341, 512, 683, 853$; $P(171) = 0.1140$, $P(0) = P(512) = 1/6$.

What the QFT does — and does not do — in Shor's algorithm

Does

- turns periodicity into frequency peaks
- decodes the binary phase pattern of phase estimation
- costs only $O(t^2)$ gates (fewer when approximated)

Does not

- compute $a^x \bmod N$ (the expensive part)
- output r directly (continued fractions needed)
- output the factors (gcds needed)

Shor = classical reduction + {superposition, modular exponentiation, inverse QFT, measurement} + classical post-processing.

Eigenvalues and eigenvectors of U_a

On the orbit $|1\rangle \to |a\rangle \to |a^2\rangle \to \cdots \to |1\rangle$, U_a is a cyclic shift of length r. Its eigenvectors are

$$|u_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k/r} \, |a^k \bmod N\rangle$$

$$U_a \, |u_s\rangle = e^{2\pi i s/r} \, |u_s\rangle \,, \qquad s = 0, \ldots, r-1$$

The order is in the eigenphases s/r.

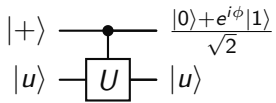
U_2 modulo 15

orbit $1 \to 2 \to 4 \to 8 \to 1$, eigenvalues $1, i, -1, -i$

Trick: no eigenvector needed

$\frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |u_s\rangle = |1\rangle$. Starting the work register in $|1\rangle$ runs phase estimation on a *uniformly random* eigenvector.

Phase kickback



$U|u\rangle = e^{i\phi}|u\rangle$: the target is **unchanged**; the phase moves to the $|1\rangle$ branch of the **control**.

- $\langle X\rangle$ of the control $= \cos\phi$: for $\phi = 2\pi/3$, $-\frac{1}{2}$.
- U^{2^j} kicks back $2^j\phi$: counting qubit j stores the phase doubled j times.
- "Controlled gates never change the control" is **false** in quantum computing.

Quantum phase estimation (QPE)

After the controlled U^{2^j} gates the counting register is

$$\bigotimes_j \frac{|0\rangle + e^{2\pi i 2^j \varphi}|1\rangle}{\sqrt{2}} = \frac{1}{\sqrt{2^t}} \sum_x e^{2\pi i x \varphi} |x\rangle$$

— exactly $\mathrm{QFT}\,|2^t\varphi\rangle$ when $2^t\varphi$ is an integer. The inverse QFT returns $y \approx 2^t\varphi$:

$$P(y) = \left| \frac{1}{2^t} \sum_x e^{2\pi i x(\varphi - y/2^t)} \right|^2$$

$t = 3$

$\varphi = 3/8$: $y = 3$ with probability 1.
$\varphi = 1/3$: $y = 3$ with $P \approx 0.69$, $y = 2$ with 0.17.

The nearest outcome always has $P \geq 4/\pi^2 \approx 0.405$; more counting qubits sharpen the peak.

Order finding = phase estimation of U_a

- eigenphase $\varphi = s/r$ with s uniform in $\{0, \ldots, r-1\}$
- measured $y \approx 2^t s/r$ — the *same* peaks as in the period-finding picture
- controlled $U_a^{2^j} = U_{a^{2^j} \bmod N}$: the constants are precomputed classically

Two views, one circuit

period finding	**phase estimation**
comb in x with spacing r	eigenphases s/r
measure work register (optional)	work register in $\|1\rangle$
Fourier peaks $k2^t/r$	$y \approx 2^t s/r$

The QPE view explains why $t \approx 2n$ counting qubits are used: the phase must be resolved to precision $\approx 1/N^2$.

Why the measurement does not give r directly

- The outcome y is **random**: we learn $y/2^t \approx s/r$ for a random s.
- s/r may not be in lowest terms: if $\gcd(s, r) > 1$ we only see a **divisor** of r.
- $y = 0$ ($s = 0$) occurs for every r and carries **no** information.
- When $r \nmid 2^t$, y is only close to $2^t s/r$; neighbouring values also occur.

Goal of post-processing

find the fraction s/r with $r \leq N$ closest to $y/2^t$, then **verify** $a^r \equiv 1$ classically — the quantum computer only proposes candidates.

Continued fractions

$$x = a_0 + \cfrac{1}{a_1 + \cfrac{1}{a_2 + \cdots}} = [a_0; a_1, a_2, \ldots]$$

Algorithm: $a_0 = \lfloor x \rfloor$, repeat on $1/(x - a_0)$. Truncations are the **convergents** — the best rational approximations.

Legendre's theorem

If $\left| \frac{y}{2^t} - \frac{s}{r} \right| < \frac{1}{2r^2}$, then s/r is a convergent of $y/2^t$. With $2^t \geq N^2$ the nearest outcome satisfies this.

$y = 171,\ t = 10,\ N = 21$

$171/1024 = [0; 5, 1, 84, 2]$
convergents: $0,\ \frac{1}{5},\ \frac{1}{6},\ \frac{85}{511}, \ldots$
largest denominator ≤ 21: $\mathbf{1/6} \Rightarrow r = 6$
check: $2^6 = 64 \equiv 1 \pmod{21}$ ✓

In Python: `Fraction(171, 1024).limit_denominator(21)` → `Fraction(1, 6)`.

Post-processing the likely outcomes for $N = 21$, $a = 2$ ILLUSTRATIVE EXPERIMENT

y	$P(y)$	$y/2^{10}$	continued fraction	s/r	r	factors
0	0.1667	0.00000	[0]	-	-	none
170	0.0285	0.16602	[0; 6; 42; 2]	1/6	6	3×7
171	0.1140	0.16699	[0; 5; 1; 84; 2]	1/6	6	3×7
341	0.1140	0.33301	[0; 3; 341]	1/3	6	3×7
342	0.0285	0.33398	[0; 2; 1; 170]	1/3	6	3×7
512	0.1667	0.50000	[0; 2]	1/2	6	3×7
683	0.1140	0.66699	[0; 1; 2; 341]	2/3	6	3×7
853	0.1140	0.83301	[0; 1; 4; 1; 84; 2]	5/6	6	3×7

Exact probabilities from the CUDA-Q state vector of the notebook's circuit ($t = 10$). $y = 512$ gives $1/2$; the **multiples test** $(2, 4, 6)$ recovers $r = 6$. $y = 0$ gives nothing. In total $P_{\text{succ}} = 0.817$ per shot.

Verifying the candidate and recovering the factors IN REPOSITORY

```
def candidate_orders(y, t, modulus, a):
    if y == 0:
        return None, []
    fraction = Fraction(y, 2 ** t)
    fraction = fraction.limit_denominator(modulus)
    base = fraction.denominator
    return fraction, [base * k for k in
        range(1, modulus // base + 1)
        if pow(a, base * k, modulus) == 1][:1]
```

Copied from `Shor.ipynb` (Section 7a).

1. denominator r' of the closest fraction
2. smallest multiple kr' with $a^{kr'} \equiv 1$ (repairs $\gcd(s, r) > 1$)
3. r even, $x = a^{r/2} \not\equiv -1$
4. $p = \gcd(x - 1, N)$, $q = \gcd(x + 1, N)$
5. check $p \cdot q = N$

Without step 2 a shot succeeds with probability about $\varphi(r)/r$.

Factoring $N = 15$ with $a = 2$: classical steps 1–4

1. $\gcd(2, 15) = 1$ — no lucky factor; continue.
2. $f(x) = 2^x \bmod 15$.
3. values:

x	0	1	2	3	4	5	6	7
f	1	2	4	8	1	2	4	8

4. period $r = 4$ — **read off here only because N is tiny**; for large N this table is impossible to compute.

Mathematical example, not a hardware run

Steps 3–4 are what the quantum computer replaces. In this lecture the quantum part was *simulated* on a CPU; no quantum device was used.

Registers (notebook): $n = 4$, $t = 8$, 12 qubits.

$N = 15$, $a = 2$: the quantum part (steps 5–6)

$$\frac{1}{16}\sum_{x=0}^{255}|x\rangle|1\rangle \xrightarrow{U} \frac{1}{16}\sum_{x}|x\rangle|2^x \bmod 15\rangle$$
$$= \frac{1}{2}\big(\,|c_1\rangle|1\rangle + |c_2\rangle|2\rangle + |c_4\rangle|4\rangle + |c_8\rangle|8\rangle\,\big)$$

with combs $|c_v\rangle = \frac{1}{8}\sum_{m=0}^{63}|x_0 + 4m\rangle$. The inverse QFT maps each comb to peaks at $y \in \{0, 64, 128, 192\}$, each with probability $1/4$.

Controlled multiplications

$2^{2^j} \bmod 15$: $2, 4, 1, 1, \ldots$ — only U_2 and U_4 are non-trivial (counting qubits x_0 and x_1).

ILLUSTRATIVE EXPERIMENT re-executed

1000 shots, seed 42: 4 distinct outcomes, success fraction 0.766 (exact 0.750).

$N = 15$, $a = 2$: post-processing and verification (steps 7–12)

y	$y/256$	fraction	candidate r	result
0	0	—	—	no information (probability 1/4)
64	0.25	1/4	4	$2^4 = 16 \equiv 1$ ✓
128	0.5	1/2	$2 \to$ multiples $\to 4$	$2^2 = 4 \neq 1$, $2^4 \equiv 1$ ✓
192	0.75	3/4	4	✓

8. verified $r = 4$: even, $2^{r/2} = 4 \not\equiv 14$
9. $\gcd(4 - 1, 15) = \gcd(3, 15) = \mathbf{3}$
10. $\gcd(4 + 1, 15) = \gcd(5, 15) = \mathbf{5}$
11. non-trivial factors 3 and 5
12. $3 \times 5 = 15$ ✓

Success per shot: 3/4 (only $y = 0$ fails).

Second example: $N = 21$, $a = 2$ ($r = 6$ does not divide 2^t)

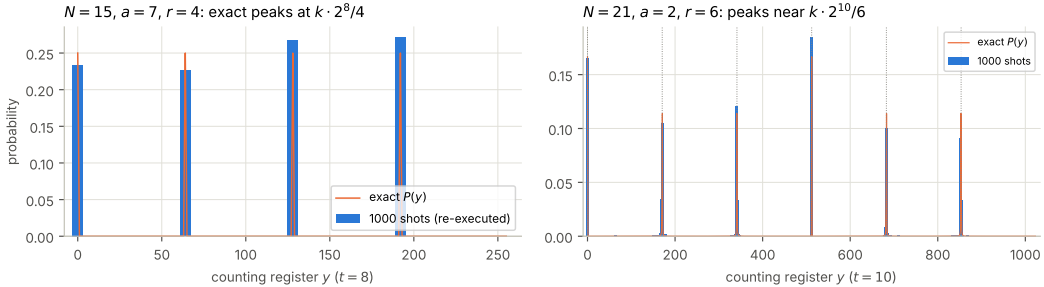
1. $\gcd(2, 21) = 1$; $n = 5$, $t = 10$, 15 qubits
2. $2^x \bmod 21$: $1, 2, 4, 8, 16, 11, 1, \ldots$
   $\Rightarrow r = 6$
3. peaks near $k \cdot 1024/6$:
   $0, 171, 341, 512, 683, 853$
4. e.g. $y = 853$: $853/1024 \approx 5/6 \Rightarrow r = 6$
5. $2^3 = 8 \not\equiv 20$: $\gcd(7, 21) = \mathbf{7}$,
   $\gcd(9, 21) = \mathbf{3}$
6. $7 \times 3 = 21$ ✓

What changes compared with $N = 15$

$r = 6$ does not divide 1024, so peaks have width: $P(171) = 0.1140$ and $P(170) = 0.0285$. Still, every likely outcome except $y = 0$ yields $r = 6$: $P_{\text{succ}} = 0.817$.

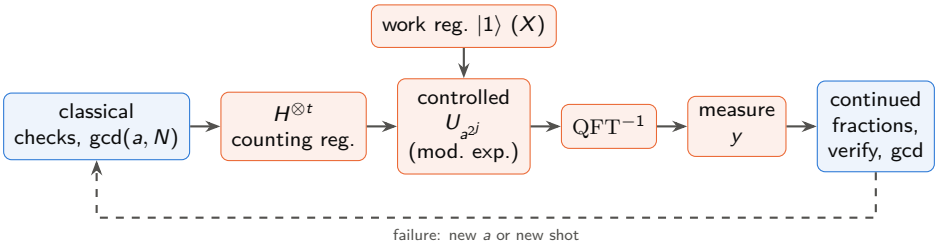
Other bases for $N = 21$: $a = 4$ ($r = 3$, odd) and $a = 5$ ($5^3 \equiv -1$) fail; $a = 6$ gives gcd = 3.

Outcome distributions RECORDED RUN ILLUSTRATIVE EXPERIMENT



Left: the recorded default run of `Shor.ipynb` ($N = 15$, $a = 7$, seed 42) — re-executed for this lecture with identical to the recorded outputs: counts $272, 268, 234, 226$ at $y = 192, 128, 0, 64$. Right: $N = 21$, $a = 2$ with the same circuit builder. CPU simulation (`qpp-cpu`); no quantum hardware.

Block diagram of Shor's algorithm



orange: quantum (simulated here); **blue**: classical. Cost is dominated by the **modular exponentiation**: $t = 2n$ controlled multiplications of n-bit numbers.

Register sizes, ancillas and scaling

- counting: $t = 2n$ qubits ($2^t \geq N^2$; minimum $\lceil 2 \log_2 N \rceil$)
- work: n qubits
- **ancillas**: real arithmetic needs extra qubits (carries, temporary results): Beauregard $2n + 3$ in total with one recycled control qubit
- idealised matrix multipliers (notebook): no ancillas, $3n$ qubits

bits n	notebook $3n$	Beauregard $2n + 3$	Gidney–Ekerå	state vector ($3n$)
4	12	11	13	65.5 kB
8	24	19	25	268 MB
16	48	35	49	10^{16} B
64	192	131	193	10^{59} B
256	768	515	773	10^{232} B
1024	3072	2051	3093	10^{926} B
2048	6144	4099	6190	10^{1851} B

Logical qubits from the notebook's formulas; state-vector memory $16 \cdot 2^{3n}$ bytes.

Modular multiplication circuits

Notebook IN REPOSITORY: each U_c is one exact $2^n \times 2^n$ permutation matrix (`mulmod_matrix`), registered as a custom gate. Fine for simulation; **not executable on hardware**.

Gate level THEORY ONLY: $|y\rangle \mapsto |cy \bmod N\rangle$ built from

- modular adders (ripple-carry or QFT-based)
- controlled multiply–accumulate, then uncompute with c^{-1}
- cost $O(n^2)$ gates per multiplier, $O(n^3)$ in total

ILLUSTRATIVE EXPERIMENT special case $N = 15$

multiplication by 2 is a cyclic **bit rotation** of the 4 work bits; by $7 = -8$: three rotations plus X on every bit (valid for $y = 1, \ldots, 14$). Controlled: controlled SWAPs (Fredkin) and CNOTs.

Fredkin $= \mathrm{CX} \cdot \mathrm{Toffoli} \cdot \mathrm{CX} = 8$ CNOTs. Such shortcuts exist only because $N = 15$ is special — they do not generalise.

Gate-level circuit for $N = 15$ ILLUSTRATIVE EXPERIMENT

$t = 3$ counting + 4 work qubits ($r = 4$ divides 2^3, so 3 counting qubits suffice here):

- controlled $\times 7$: 9 Fredkins + 4 CNOTs = 76 CNOTs
- controlled $\times 4$: 6 Fredkins = 48 CNOTs
- inverse QFT: 1 SWAP (3 CNOTs) + 3 controlled phases

Total 127 CNOTs, 142 one-qubit gates, 273 gates (CUDA-Q `estimate_resources`).

Checked

the gate-level multipliers equal the notebook's permutation matrices on all relevant inputs, and the counting distribution $(\frac{1}{4}, 0, \frac{1}{4}, 0, \frac{1}{4}, 0, \frac{1}{4}, 0)$ agrees with the matrix-gate circuit to $1.5e-15$.

The multiplier constants use the known order (they skip $c = 1$); this is a *teaching* circuit, not evidence of a speed-up.

Gate count, depth and connectivity

Recorded RECORDED RUN ($N = 15$, $t = 8$, 12 qubits):

- 51 gates without measurement: 8+8 H, 1 X, 4 SWAP, 28 `cr1`, 2 controlled custom multipliers
- depth 20

These counts treat a whole multiplier as *one* gate — real circuits are much larger.

Connectivity

Controlled multipliers couple every counting qubit with the whole work register; on hardware with nearest-neighbour connectivity SWAP networks add depth. Surface-code architectures use lattice surgery for long-range gates.

As N grows: qubits $\propto n$, gates $\propto n^3$ (schoolbook), depth $\propto n^3$ or less with parallel arithmetic.

Environment and framework

- **CUDA-Q** 0.16.0 (Python), NumPy, `fractions`, `math` — all installed and used.
- `BACKEND` in `Shor.ipynb`: `auto`, `cpu`, `gpu`, `multi-gpu` (state-vector or density-matrix simulators); `hardware` is **refused**. The recorded run used `qpp-cpu`.
- **Qiskit / Qiskit Aer**: not installed in this repository and not used. The algorithm itself is framework-independent; a Qiskit version would need its own gate-level or matrix-based multipliers.

Notebook parameters

`N_TO_FACTOR`	15
`BASE_A`	7 (`None`: random)
`NUM_COUNTING_QUBITS`	`None` ($2n$)
`SHOTS`	1000
`SEED`	42
`MAX_SIMULATION_QUBITS`	26

Circuit creation: registers, Hadamards, modular exponentiation IN REPOSITORY

```
def mulmod_matrix(c, modulus, n): # |y> -> |c*y mod N>, identity for y >= N
    dim = 2 ** n
    matrix = np.zeros((dim, dim))
    for y in range(dim):
        matrix[(c * y) % modulus if y < modulus else y, y] = 1.0
    return matrix

kernel = cudaq.make_kernel()
count = kernel.qalloc(t) # counting register
work = kernel.qalloc(n) # work register
kernel.x(work[n - 1]) # |1>: last qubit = least significant bit
kernel.h(count) # uniform superposition
for j in range(t):
    c = pow(a, 2 ** j, modulus) # a^(2^j) mod N, classical repeated squaring
    if c == 1:
        continue # identity: nothing to apply
    name = f"mulmod_{c}_mod_{modulus}"
    if name not in registered_ops:
        cudaq.register_operation(name, mulmod_matrix(c, modulus, n))
        registered_ops.add(name)
    getattr(kernel, name)(count[t - 1 - j], # first argument = control
                          *[work[i] for i in range(n)])
```

Abridged from `build_shor_kernel` in `Shor.ipynb`. `count[0]` is the most significant bit of y.

Inverse QFT, measurement and sampling IN REPOSITORY

```
for j in range(t // 2):
    kernel.swap(count[j], count[t - 1 - j])
for jj in range(t):
    j = t - 1 - jj
    for kk in range(t - 1 - j):
        k = t - 1 - kk
        kernel.cr1(-np.pi / 2 ** (k - j),
                   count[k], count[j])
    kernel.h(count[j])
kernel.mz(count) # counting register only

counts = cudaq.sample(kernel, shots_count=SHOTS)
for bits, c in counts.items():
    y = int(bits, 2) # MSB first
    fraction, orders = candidate_orders(y, t, N, a)
```

- `cr1`: controlled $R_1(\lambda)$ with negative angles (inverse QFT)
- only the counting register is measured
- each distinct y is post-processed; then gcd and the check $p \cdot q = N$
- if no shot yields factors, a new random a is tried

Factor verification and reproducibility

- The notebook compares the result with **trial division** and the order with a brute-force `classical_order` (feasible only for tiny N).
- Final report: "`Verification p x q == N : CORRECT`", "`Order r found / classical check : 4 / 4`".
- Fixed `SEED = 42`: re-running gives the same counts (identical to the recorded outputs).

Independent checks for this package

`make_figures.py` re-executes the notebook's builder; `verify_answers.py` recomputes all exam answers with exact arithmetic and compares CUDA-Q state vectors for $N = 15, 21, 33, 39$ with a NumPy formula for $P(y)$.

Kinds of "running" Shor's algorithm

kind	what is computed	in this package
mathematical	orders, peaks, probabilities by hand / exact formulas	worked examples
state vector	all 2^q amplitudes, exact $P(y)$	ILLUSTRATIVE EXPERIMENT (`get_state`)
shot-based	samples y from the circuit	RECORDED RUN ILLUSTRATIVE EXPERIMENT (`sample`)
ideal	no errors	all of the above
noisy	density matrix with a noise model	ILLUSTRATIVE EXPERIMENT gate-level $N = 15$
hardware	a physical quantum processor	**not performed**

Simulation proves that the *circuit* is correct; it does not demonstrate any quantum speed-up — the simulator itself does exponential work.

Recorded run of `Shor.ipynb` RECORDED RUN

CUDA-Q / target	0.16.0 / `qpp-cpu` (fp64)
GPUs visible	0
N, a	15, 7
qubits	8 counting + 4 work = 12
gates / depth	51 (+ 8 `mz`) / 20
shots	1000
counts y: n_y	192: 272, 128: 268, 0: 234, 64: 226
success fraction	0.766 (exact 0.75)
result	$15 = 3 \times 5$, $r = 4$ (checked)

construction	0.0029 s
compilation (MLIR + JIT)	0.0353 s
quantum execution	0.0179 s
whole cell	0.073 s
peak host RAM	0.87 GB
process RSS	0.55 GB
state vector	$16 \cdot 2^{12} = 65.5$ kB

Times are for one laptop-class CPU run and vary between runs and machines; they are not a benchmark of quantum hardware.

Further cases with the notebook's circuit ILLUSTRATIVE EXPERIMENT

N	a	r	qubits	U gates	distinct y (1000 shots)	P_{succ} exact	from 1000 shots	factors
15	7	4	$8+4=12$	2	4	0.750	0.766	3×5
15	2	4	$8+4=12$	2	4	0.750	0.766	3×5
15	14	2	$8+4=12$	1	2	0.000	0.000	none
21	2	6	$10+5=15$	10	54	0.817	0.822	3×7
35	3	12	$12+6=18$	12	76	0.904	0.897	5×7

P_{succ}: probability that one shot yields the factors (exact, from the state vector) and the fraction observed in 1000 shots (seed 42). $a=14$ **always fails** ($14\equiv-1$). $N=35$ (18 qubits) took 5.64 s on the CPU.

Noisy simulation of the gate-level $N = 15$ circuit ILLUSTRATIVE EXPERIMENT

p	$y=0$	$y=1$	$y=2$	$y=3$	$y=4$	$y=5$	$y=6$	$y=7$	P_{succ}
0	0.250	0.000	0.250	0.000	0.250	0.000	0.250	0.000	0.750
0.001	0.256	0.001	0.243	0.001	0.253	0.001	0.243	0.001	0.740
0.003	0.262	0.003	0.234	0.003	0.257	0.003	0.234	0.003	0.726
0.01	0.258	0.010	0.225	0.010	0.254	0.010	0.224	0.010	0.703
0.03	0.226	0.027	0.220	0.027	0.226	0.027	0.220	0.027	0.665

`density-matrix-cpu`; depolarizing channel p after every H, R_z, X and two-qubit depolarizing p after every CNOT and controlled phase (127 CNOTs). Success falls from 0.750 towards the random-outcome baseline 0.375: 0.703 at $p = 0.01$, 0.665 at $p = 0.03$. An idealised noise model — not a model of a specific device.

Approximate inverse QFT for $N = 21$ ILLUSTRATIVE EXPERIMENT

max. $k - j$ kept	controlled phases	P_{succ}
1	9	0.4397
2	17	0.6892
3	24	0.7865
4	30	0.8105
9	45	0.8165

Drop controlled phases between qubits at distance $k - j > m$ (angles $\leq \pi/2^m$).

- $m = 4$: 30 instead of 45 phases, success 0.8105 vs 0.8165
- $m = 1$: only 9 phases, success drops to 0.44

Asymptotically $m \approx \log_2 t$ suffices, reducing the QFT to $O(t \log t)$ gates. The QFT was never the bottleneck — modular exponentiation is.

Practical challenges

- **qubits**: $\approx 2n$–$3n$ *logical* qubits; ≈ 6200 for RSA-2048
- **gates**: billions of Toffoli gates for RSA-2048
- **error correction**: each logical qubit $\approx$ hundreds to thousands of physical qubits
- **fidelity and decoherence**: physical error rates $\sim 10^{-3}$ must be suppressed below $\approx 1/(\text{number of operations}) \sim 10^{-10}$ per logical operation
- **depth and connectivity**: long sequential arithmetic, routing overhead
- **classical control**: real-time decoding of error syndromes

Small demonstrations $\neq$ RSA

Factoring 15 or 21 uses a handful of qubits and often a circuit simplified with *the known answer* (as in our gate-level $N = 15$ circuit). Such "compiled" demonstrations do not scale and say nothing about RSA-2048.

Classical simulation is no way out: the notebook's memory budget of 80 GB allows only $N \leq 1023$ (30 qubits).

Why simulation and today's hardware stop early

Simulation IN REPOSITORY (notebook's reverse estimation):

- 80 GB $\Rightarrow$ 32-qubit state vector $\Rightarrow$ $N \leq 1023$ (10 bits)
- 100 logical qubits $\Rightarrow$ 33-bit N ($3n$) or 48-bit N ($2n + 3$)
- each extra bit: 3 more qubits, $8\times$ memory

Hardware THEORY ONLY:

- current devices: noisy physical qubits, no large-scale error correction yet
- gap to RSA-2048: millions of high-quality physical qubits running for hours to days

No hardware claims are made in this lecture; the notebook refuses hardware targets.

How Shor's algorithm threatens public-key cryptography

scheme	hard problem	quantum attack
RSA	factoring	Shor: broken (polynomial)
Diffie–Hellman, DSA	discrete log in $\mathbb{Z}_p^*$	Shor: broken
ECDH, ECDSA, EdDSA	elliptic-curve discrete log	Shor: broken (fewer qubits than RSA of equal strength)
AES, SHA-2/3	no algebraic period	Grover: square-root speed-up only $\Rightarrow$ use 256-bit keys

Theory versus practice

Theoretical impact: complete break of RSA/ECC by a large fault-tolerant quantum computer.
Practical capability today: none — no such machine exists. The risk is about the *future*.

Post-quantum cryptography and NIST standards

Post-quantum cryptography: classical algorithms based on problems with no known efficient quantum algorithm (lattices, hash functions, codes).

- **FIPS 203** ML-KEM (Kyber) — key encapsulation
- **FIPS 204** ML-DSA (Dilithium) — signatures
- **FIPS 205** SLH-DSA (SPHINCS+) — hash-based signatures

published by NIST in August 2024; HQC (code-based KEM) selected in 2025 as a backup.

Not to confuse

Post-quantum cryptography runs on classical computers. **Quantum key distribution** is a different technology that needs quantum hardware.

Quantum-safe migration: why organisations act now

- **Harvest now, decrypt later**: encrypted traffic recorded today can be decrypted once a large quantum computer exists.
- **Mosca's inequality**: act if (data lifetime) + (migration time) > (time until a cryptographically relevant quantum computer).
- Migrations take years: inventory, hybrid key exchange (e.g. X25519 + ML-KEM), crypto-agility.

Real-world implications

TLS, VPNs, code signing, PKI certificates, long-lived devices. Government roadmaps (e.g. NIST IR 8547, draft 2024) plan to deprecate quantum-vulnerable public-key algorithms by 2030 and disallow them by 2035.

Order finding, phase estimation and the hidden subgroup problem

- Order finding is an instance of the **abelian hidden subgroup problem**: find the subgroup $r\mathbb{Z}$ on which $f(x) = a^x$ is constant.
- The same Fourier-sampling method solves **discrete logarithms** (two-dimensional period of $g^x h^{-y}$) and Simon's problem.
- **Ekerå–Håstad** variants factor RSA moduli via a short discrete logarithm with fewer qubits; **Regev (2023)** proposed a multidimensional variant with $\approx n^{1.5}$ gates per run but more runs and qubits.
- Non-abelian generalisations (graph isomorphism, lattice problems) have **no** known efficient algorithm — the basis of post-quantum security.

Approximate and semiclassical QFT

Approximate QFT (Coppersmith 1994): omit rotations by angles $< \pi/2^m$; $O(t \log t)$ gates, small error (see Module 13).

Semiclassical QFT (Griffiths–Niu 1996): measure the counting qubits one at a time and apply the remaining rotations **classically controlled**. One physical control qubit is **recycled** $2n$ times.

Consequence

qubits $3n \to 2n + 3$ (Beauregard 2003: n work, $n + 1$ for a QFT-based adder, 1 ancilla, 1 control). Same measurement statistics; needs mid-circuit measurement and fast feed-forward.

Fault tolerance: logical versus physical qubits

- **Physical qubit**: noisy hardware element (error $\sim 10^{-3}$).
- **Logical qubit**: encoded in many physical qubits, e.g. a **surface code** patch of distance d ($\approx 2d^2$ qubits).
- Logical error $\sim (p/p_{\text{th}})^{(d+1)/2}$: grows exponentially better with d below threshold $p_{\text{th}} \approx 1\%$.
- Clifford gates are cheap; T/Toffoli gates need **magic states** (distillation or cultivation) — they dominate Shor's cost.

Published RSA-2048 estimates

THEORY ONLY

	physical qubits	time
Gidney–Ekerå 2019	≈ 20 million	≈ 8 h
Gidney 2025	< 1 million	< 1 week

assumptions: 10^{-3} physical error rate, $1\,\mu$s surface-code cycle. Estimates, not experiments.

Optimised arithmetic and research directions

Optimised modular exponentiation

- windowed arithmetic and table lookups
- coset representation, oblivious carry runways
- approximate residue arithmetic (Gidney 2025)
- Montgomery and Karatsuba-style multipliers

Open directions

- better codes (yoked surface codes, qLDPC codes)
- cheaper magic states
- elliptic-curve discrete logs (fewer qubits than RSA)
- honest benchmarks: avoid "compiled" demonstrations

The trend: each new paper lowers the estimated cost — one reason to migrate to post-quantum cryptography early.

Common misconceptions

1. "Shor tries all factors in parallel" — **no**: one measurement gives one sample; interference does the work.
2. "The QFT factors N" — the QFT is a subroutine; modular exponentiation and classical steps are essential.
3. "The measured y is the period" — $y/2^t \approx s/r$; continued fractions and verification are needed.
4. "$y = 0$ means $r = 1$" — $y = 0$ occurs for every r and is uninformative.
5. "A correct order always gives factors" — odd r or $a^{r/2} \equiv -1$ fail; repeat.
6. "Shor works for $N = p^k$" — prime powers are handled classically.
7. "Factoring 15 on a device shows RSA is broken" — tiny compiled circuits do not scale.
8. "Simulating Shor demonstrates quantum speed-up" — the simulator does exponential classical work.

Summary

1. factoring $\rightarrow$ order finding (classical reduction)
2. order finding = phase estimation of U_a
3. Hadamards, controlled $U_{a^{2^j}}$, inverse QFT, measure
4. $y/2^t \approx s/r \rightarrow$ continued fractions $\rightarrow$ verify r
5. $\gcd(a^{r/2} \pm 1, N) \rightarrow$ factors; repeat on failure

Numbers to remember

$N = 15$: 12 qubits, outcomes $0, 64, 128, 192$, success $3/4$
$N = 21$: 15 qubits, $r = 6$, success ≈ 0.82
RSA-2048: ≈ 6200 logical, $< 10^6$–2×10^7 physical qubits (estimates)
Post-quantum: FIPS 203/204/205

Syllabus coverage map (1/2)

Module	Slides
1 Introduction	what it is 4, history 5, factoring vs prime factorisation 6, classical vs quantum 7
2 Mathematical foundations	primes, gcd, Euclid 8; modular arithmetic 9; totient, order 10; modular exponentiation 11; examples 12
3 Classical factorization	trial division 13; complexity 14; RSA 15
4 Core idea	reduction 16; $f(x) = a^x \bmod N$ 17; conditions on r 18; exceptions 19
5 Quantum foundations	registers 20; superposition, parallelism 21; gates, reversibility 22; entanglement, measurement, interference 23
6 Period finding	circuit 24; states 25; exact peaks 26; intuition 27
7 QFT	definition 28; circuit, inverse 29; peaks 30; role 31
8 Phase estimation	eigenvectors 32; kickback 33; QPE 34; QPE = order finding 35

Syllabus coverage map (2/2)

Module	Slides
9 Continued fractions	why needed 36; algorithm 37; worked table 38; verification, gcd 39
10 Worked examples	$N = 15$ steps 1–4 40, 5–6 41, 7–12 42; $N = 21$ 43; distributions 44
11 Circuit architecture	blocks 45; sizes, ancillas 46; multipliers 47; gate level 48; depth, connectivity 49
12 Implementation	environment 50; circuit code 51; QFT, sampling 52; verification 53
13 Simulation, performance	kinds 54; recorded run 55; cases 56; noise 57; approximate QFT 58
14 Limitations	challenges 59; scaling 60
15 Cryptography	threat 61; PQC 62; migration 63
16 Advanced topics	HSP, variants 64; approximate/semiclassical QFT 65; fault tolerance 66; research 67

References

- P. W. Shor, Algorithms for quantum computation: discrete logarithms and factoring, FOCS 1994; SIAM J. Comput. 26 (1997) 1484.
- M. A. Nielsen, I. L. Chuang, *Quantum Computation and Quantum Information*, Ch. 5 (QFT, phase estimation, order finding).
- D. Coppersmith, An approximate Fourier transform useful in quantum factoring, IBM RC19642 (1994).
- R. B. Griffiths, C.-S. Niu, Semiclassical Fourier transform for quantum computation, PRL 76 (1996) 3228.
- S. Beauregard, Circuit for Shor's algorithm using $2n + 3$ qubits, QIC 3 (2003) 175.
- C. Gidney, M. Ekerå, How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits, Quantum 5 (2021) 433.
- C. Gidney, How to factor 2048 bit RSA integers with less than a million noisy qubits, arXiv:2505.15917 (2025).
- J. A. Smolin, G. Smith, A. Vargo, Oversimplifying quantum factoring, Nature 499 (2013) 163.
- NIST FIPS 203, 204, 205 (2024). NVIDIA CUDA-Q documentation, version 0.16.0.